

UA1 Projet1 G4 Les_Hadoop_Riders

UA1 – Projet 1 : Cluster Hadoop avec Docker

Groupe G4 – Les_Hadoop_Riders

Image Docker Hub	leshadoopriders/hadoop-tp-g4:1.0
Lien	https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4
Remise	4 octobre 2026
Présentation	8 octobre 2026

Équipe : Komla Petro Asinyo, Kassoum Dene, Joel Kazoni Tugirimana, Forbes Magène, Frank A Simo Ngounou, Wren Surprenant-Nicolson

On s'est basés sur le dépôt du cours (hadoop-cluster-docker), puis on a adapté la config pour avoir un vrai cluster avec 1 master et 5 workers, comme demandé dans l'énoncé.

Partie 1 – Théorie

1. Cas d'usage Big Data (e-commerce)

On a choisi l'e-commerce, parce que ça colle bien avec notre fichier `transactions.csv` (ventes par magasin).

Imaginons une chaîne de magasins qui reçoit des milliers de transactions par jour (caisse + site web). Le problème, c'est pas juste « stocker un Excel » :

- **Volume** : l'historique sur plusieurs années devient énorme (transactions, logs de navigation, stocks).
- **Vélocité** : les données arrivent en continu. Si on attend le lendemain pour analyser, on rate des alertes (fraude, rupture de stock).
- **Variété** : on a du CSV, du JSON, des logs texte, parfois des images produits. Ce n'est pas un seul format propre.

Dans ce contexte, HDFS sert à stocker tout ça sur plusieurs machines, et YARN à lancer des traitements dessus sans tout faire sur un seul PC.

2. HDFS vs système de fichiers classique

Sur Windows/Linux classique (NTFS, ext4), un fichier est sur **un** disque / **une** machine. Si le disque lâche et qu'on n'a pas de backup, c'est fini.

Sur HDFS, le fichier est découpé en **blocs**. Ces blocs sont copiés sur plusieurs DataNodes (réplication). Le NameNode garde la « carte » : quel bloc est où. Du coup :

- on peut lire en parallèle ;
- si un worker tombe, les autres ont encore une copie.

C'est ça la grosse différence : local = centralisé ; HDFS = distribué + répliqué.

3. ResourceManager vs NodeManager

Quand on lance un job (ex. calcul de pi) :

- Le **ResourceManager** (sur le master) regarde les ressources du cluster et décide où placer les tâches.
- Le **NodeManager** (sur chaque worker) exécute vraiment les conteneurs sur sa machine et dit au RM si ça va ou pas.

En gros : le RM organise, les NM font le travail.

Partie 2 – Pratique

A. Docker : master + 5 workers

On a construit une image et lancé 6 conteneurs avec Docker Compose.

```
docker compose up --build -d
docker ps
```

Conteneurs : hadoop-master, hadoop-worker1 ... hadoop-worker5

Réseau : hadoop-net

Ports ouverts sur la machine : 9870 (HDFS UI), 8088 (YARN), 9000 (HDFS).

Capture : captures/00-docker-ps.txt et captures d'écran dans captures/.

Pour vérifier les DataNodes :

```
docker exec -it hadoop-master hdfs dfsadmin -report
```

On obtient bien **Live datanodes (5)**.

UI : http://localhost:9870

HadoopOverviewDatanodesDatanode Volume FailuresSnapshotStartup ProgressUtilities

Overview

'hadoop-master:9000' (active)

Started:

Sat Oct 03 11:02:46 -0400 2026

Version:

3.3.6, r1be78238728da9266a4f8b195058f08fd012bf9c

Compiled:

Sun Jun 18 04:22:00 -0400 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)

Cluster ID:

CID-c8f2f423-c208-4cdf-9c10-5573319713da

Block Pool ID:

BP-329230766-172.23.0.2-1791033373712

Summary

Security is off.
Safemode is off.
32 files and directories, 9 blocks (9 replicated blocks, 0 erasure coded block groups) = 41 total filesystem object(s).
Heap Memory used 89.9 MB of 242 MB Heap Memory. Max Heap Memory is 2.4 GB.
Non Heap Memory used 69.58 MB of 72.25 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	4.92 TB
Configured Remote Capacity:	0 B
DFS Used:	2.18 MB (0%)
Non DFS Used:	90.97 GB
DFS Remaining:	4.58 TB (93.11%)
Block Pool Used:	2.18 MB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	5 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0

NameNode

HadoopOverviewDatanodesDatanode Volume FailuresSnapshotStartup ProgressUtilities

Datanodes

Datanode Information

In service

Down

Decommissioning

Decommissioned

Decommissioned & dead

Entering Maintenance

In Maintenance

In Maintenance & dead

Datanode usage histogram

Disk usage of each DataNode (%)	Count
0	9

In operation

DataNode State: All

Show 25 entries

Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Version
✓ /default-rack/hadoop-worker1:9866 (172.23.0.5:9866)	http://hadoop-worker1:9866	2s	258m	604 KB	18.19 GB	1006.85 GB	7	604 KB (0%)	3.3.6
✓ /default-rack/hadoop-worker3:9866 (172.23.0.4:9866)	http://hadoop-worker3:9866	2s	354m	344.99 KB	18.19 GB	1006.85 GB	3	344.99 KB (0%)	3.3.6
✓ /default-rack/hadoop-worker4:9866 (172.23.0.7:9866)	http://hadoop-worker4:9866	2s	263m	353.43 KB	18.19 GB	1006.85 GB	5	353.43 KB (0%)	3.3.6
✓ /default-rack/hadoop-worker5:9866 (172.23.0.3:9866)	http://hadoop-worker5:9866	2s	276m	178.31 KB	18.19 GB	1006.85 GB	5	178.31 KB (0%)	3.3.6
✓ /default-rack/hadoop-worker2:9866 (172.23.0.6:9866)	http://hadoop-worker2:9866	2s	278m	667.16 KB	18.19 GB	1006.85 GB	6	667.16 KB (0%)	3.3.6

DataNodes

Publication Docker Hub

Compte créé : **leshadoopridders**

docker login
docker push leshadoopridders/hadoop-tp-g4:1.0

Lien public : <https://hub.docker.com/r/leshadoopridders/hadoop-tp-g4>

Petit détail : Docker refuse les majuscules dans le nom d'image, donc on a mis g4 et pas G4.

B. Manipulations HDFS

Création du dossier, droits, et upload du CSV (fichier préparé dans data/transactions.csv) :

```
hdfs dfs -mkdir -p /data/ventes/2026
hdfs dfs -chmod 755 /data/ventes/2026
hdfs dfs -put -f /data/transactions.csv /data/ventes/2026/
hdfs dfs -chmod 644 /data/ventes/2026/transactions.csv
hdfs dfs -ls -d /data/ventes/2026
hdfs dfs -ls /data/ventes/2026/transactions.csv
hdfs fsck /data/ventes/2026/transactions.csv -files -blocks -locations
hdfs dfs -stat "%n | taille=%b | replication=%r | block_size=%o"
/data/ventes/2026/transactions.csv
```

Après chmod, on voit bien drwxr-xr-x sur le dossier et -rw-r--r-- sur le fichier (preuve : captures/12-droits-chmod.txt).

Ce qu'on a vu chez nous : - taille ≈ 377 octets - réplication de départ = 3 - 1 seul bloc (normal, le fichier est petit ; la taille de bloc par défaut est 128 Mo) - fsck : HEALTHY

Lecture des 5 premières lignes **depuis HDFS** (sans re-télécharger le fichier à la main) :

```
hdfs dfs -cat /data/ventes/2026/transactions.csv | head -n 5
```

Ensuite on a ajouté transactions_part2.csv, puis fusionné avec getmerge :

```
hdfs dfs -put -f /data/transactions_part2.csv /data/ventes/2026/
hdfs dfs -getmerge /data/ventes/2026 /tmp/transactions_merged.csv
hdfs dfs -put -f /tmp/transactions_merged.csv
/data/ventes/2026/transactions_merged.csv
```

Pour la suppression / corbeille :

```
hdfs dfs -rm /data/ventes/2026/transactions_part2.csv
# pour forcer la corbeille :
hdfs dfs -D fs.trash.interval=10080 -rm /data/ventes/2026/to_delete.csv
hdfs dfs -ls -R /user/hadoop/.Trash
# suppression définitive :
# hdfs dfs -rm -skipTrash ...
```

Au début, la corbeille n'était pas activée (intervalle à 0), donc le fichier disparaissait direct. En mettant fs.trash.interval, on a bien vu le fichier arriver dans .Trash.

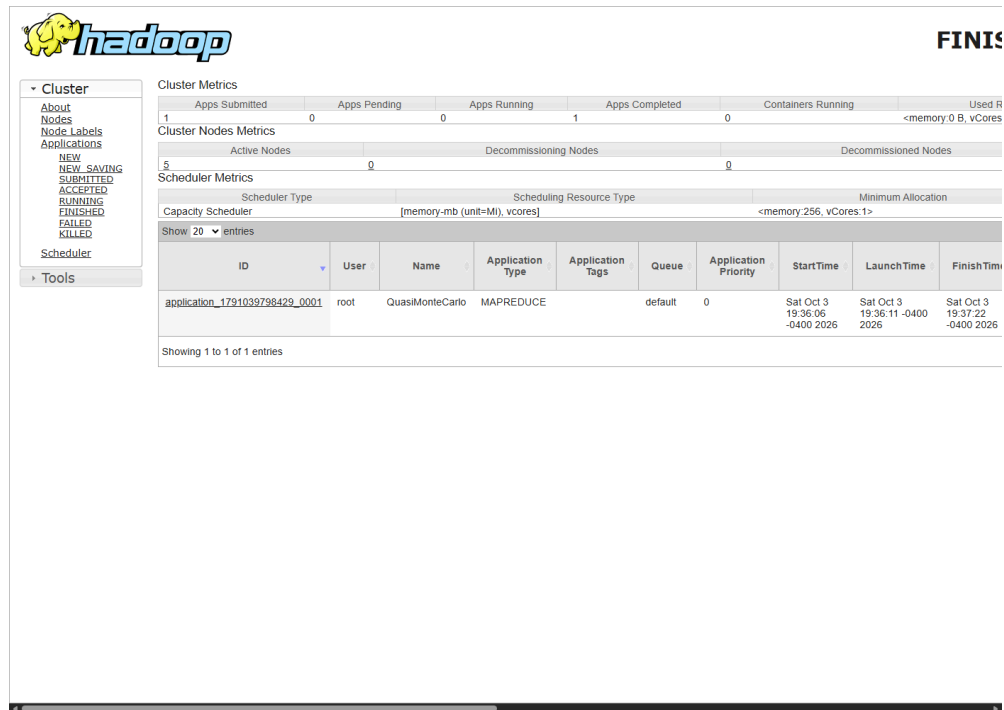
Preuves terminal : captures/01-demo-hdfs.txt, captures/10-trash.txt.

C. Job YARN – calcul de pi

On a choisi l'exemple pi (Monte Carlo), comme dans l'énoncé :

```
yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar pi 4 1000
```

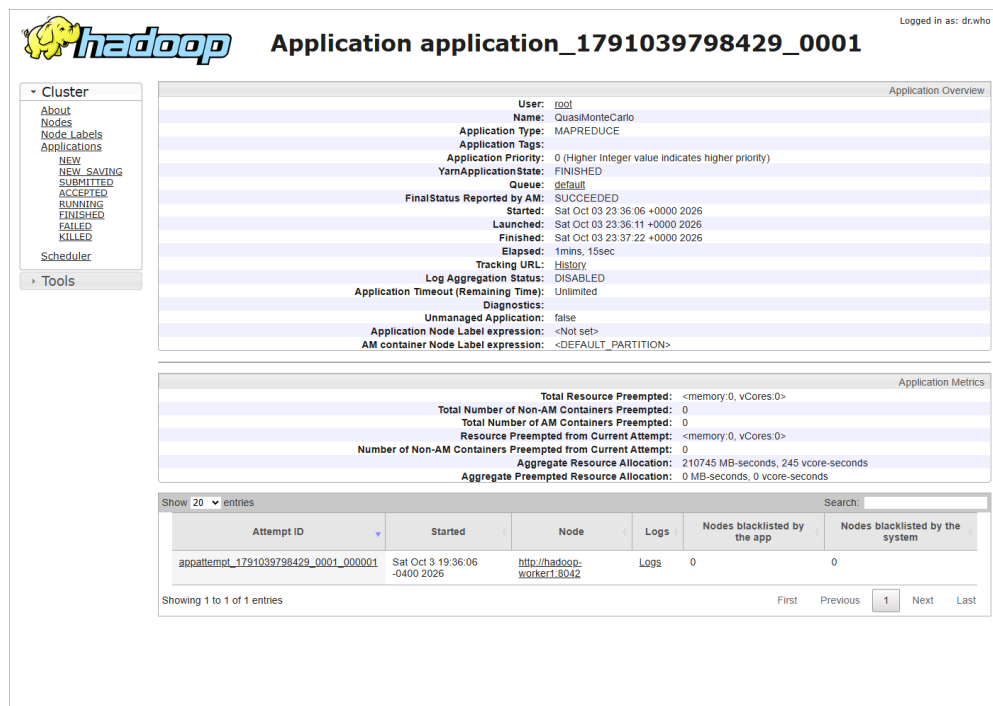
Résultat : - Application : application_1791039798429_0001 - Nom : QuasiMonteCarlo -
État final : **SUCCEEDED** - Estimation de $\pi \approx 3.14$ - 4 maps, puis reduce à 100%



The screenshot shows the Hadoop UI Cluster Metrics page. The top navigation bar includes the Hadoop logo and the word "FINIS". The left sidebar contains a "Cluster" menu with options like "About", "Nodes", "Node Labels", "Applications", "NEW", "NEW SAVING", "SUBMITTED", "ACCEPTED", "RUNNING", "FINISHED", "FAILED", "KILLED", "Scheduler", and "Tools". The main content area displays "Cluster Metrics" with a table showing 1 App Submitted, 0 Apps Pending, 0 Apps Running, 1 Apps Completed, 0 Containers Running, and Used B. Below this is a "Cluster Nodes Metrics" section with a table showing 5 Active Nodes, 0 Decommissioning Nodes, and 0 Decommissioned Nodes. The "Scheduler Metrics" section shows a table with columns for ID, User, Name, Application Type, Application Tags, Queue, Application Priority, Start Time, Launch Time, and Finish Time. The table contains one entry for application_1791039798429_0001, which is in the "FINISHED" state.

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	Start Time	Launch Time	Finish Time
application_1791039798429_0001	root	QuasiMonteCarlo	MAPREDUCE		default	0	Sat Oct 3 19:36:06 -0400 2026	Sat Oct 3 19:36:11 -0400 2026	Sat Oct 3 19:37:22 -0400 2026

YARN



The screenshot shows the Hadoop UI Application Overview page for application_1791039798429_0001. The top navigation bar includes the Hadoop logo and the word "FINIS". The left sidebar contains a "Cluster" menu with options like "About", "Nodes", "Node Labels", "Applications", "NEW", "NEW SAVING", "SUBMITTED", "ACCEPTED", "RUNNING", "FINISHED", "FAILED", "KILLED", "Scheduler", and "Tools". The main content area displays "Application Overview" with a table showing details for the application, including User (root), Name (QuasiMonteCarlo), Application Type (MAPREDUCE), Application Tags, Application Priority, YarnApplicationState (FINISHED), Queue (default), FinalStatus Reported by AM (SUCCEEDED), Started (Sat Oct 3 23:36:06 +0000 2026), Launched (Sat Oct 3 23:36:11 +0000 2026), Finished (Sat Oct 3 23:37:22 +0000 2026), Elapsed (1mins, 15sec), Tracking URL (History), Log Aggregation Status (DISABLED), Application Timeout (Remaining Time) (Unlimited), Diagnostics (Unmanaged Application: false), Application Node Label expression (<Not set>), and AM container Node Label expression (<DEFAULT_PARTITION>). Below this is an "Application Metrics" section with a table showing Total Resource Preempted, Total Number of Non-AM Containers Preempted, Total Number of AM Containers Preempted, Resource Preempted from Current Attempt, Number of Non-AM Containers Preempted from Current Attempt, Aggregate Resource Allocation, and Aggregate Preempted Resource Allocation. The table contains one entry for application_1791039798429_0001, which is in the "FINISHED" state.

Attempt ID	Started	Node	Logs	Nodes blacklisted by the app	Nodes blacklisted by the system
appattempt_1791039798429_0001_000001	Sat Oct 3 19:36:06 -0400 2026	http://hadoop-worker1.8042	Logs	0	0

Job SUCCEEDED

Après le job, on a eu un warning parce que le JobHistory Server (port 10020) n'était pas lancé. Le job était quand même réussi dans l'UI YARN (8088), donc on a gardé ça comme point de troubleshooting.

Partie 3 – Monitoring et analyse

Interfaces web

NameNode (9870)

On voit l'espace disque (chez nous ça affiche une grosse capacité à cause de Docker/WSL) et surtout les **5 DataNodes live**. Pas de missing blocks sur nos tests.

YARN (8088)

On retrouve l'appli pi en FINISHED / SUCCEEDED, avec le temps et la mémoire consommée (environ 198589 MB-seconds et 216 vcore-seconds).

Changement du facteur de réplication

```
hdfs dfs -setrep -w 2 /data/ventes/2026/transactions.csv
```

Après la commande : replication = 2.

Hadoop met à jour la cible côté NameNode. Comme on descendait de 3 à 2, il a juste enlevé une copie en trop sur les DataNodes. L'option -w attend que ce soit fini avant de rendre la main.

Problèmes qu'on a eus (et comment on a réglé)

1. **Compose essayait de pull l'image sur Docker Hub alors qu'elle n'existait pas encore**
Erreur du style pull access denied.
Fix : pull_policy: never + build local d'abord.
 2. **Nom d'image avec majuscules (G4)**
Docker refuse.
Fix : leshadoopriders/hadoop-tp-g4:1.0.
 3. **Build trop long** (téléchargement Hadoop depuis archive.apache.org très lent)
On est partis sur l'image apache/hadoop:3.3.6 et on a mis nos configs + entryptpoint par-dessus pour avoir master/workers.
-

Annexes

Commandes utiles

```
docker compose up --build -d
docker exec -it hadoop-master bash
hdfs dfsadmin -report
```

```
yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-  
3.3.6.jar pi 4 1000  
docker push leshadoopridders/hadoop-tp-g4:1.0
```

Contenu du dossier rendu

- Dockerfile, docker-compose.yml, entrypoint.sh
- config/ (core-site, hdfs-site, yarn-site, mapred-site, workers)
- data/ (CSV)
- scripts/ (demos)
- captures/ (preuves)
- ce rapport

Qui présente quoi (8 octobre)

À répartir dans le groupe (exemple) : 1. Cas e-commerce / 3V
2. Architecture Docker + preuve des 5 DataNodes
3. Commandes HDFS
4. Job pi + UI YARN
5. setrep + monitoring
6. Troubleshooting + lien Docker Hub

Présence obligatoire le 8 octobre.